ARGUS Demo Runbook

Running the demo end to end

ARGUS demo runbook

Capture → classify → store → label → federate → update, on the prototype node or on a laptop against a recorded video.

What changed on 2026-09-14

Change	Why it matters for the demo
Face recognition deprecated — off by default, nothing loads for it	Measured at 12–27× slower frame rate with it on. `insightface` is no longer in `requirements.txt`, so the Pi install no longer compiles it.
Detection runs continuously in the background	Before, nothing was detected unless someone had the video stream open. Now events are collected whether or not anyone is watching (FR-19).
Only meaningful activity is stored	One event per tracked object (not per frame); sustained motion becomes one `motion` event; flicker is discarded.
Federated updates go through a validation gate	The averaged model replaces the live one only if it doesn't lower balanced accuracy on this node's own held-out samples. It persists across restarts.
Faster CPU path	Motion runs on a 320 px copy; YOLO filters to ARGUS classes; NCNN (Pi) and OpenVINO (x86) exports load automatically from `models/`.
Video-file camera + isolated data dir	The whole node can run against a recording with a throwaway database.

A · Mock demo on a laptop (no Pi needed)

Everything real except the camera (a looping video) and the database (`data/mock`, isolated from real data).

Terminal 1 — node

```
cd argus
python scripts/mock_demo.py --reset
```

`--reset` wipes `data/mock` and creates the admin account with password `argus-demo`. Use `--source 0` for a webcam, `--threat` to add the threat stage. Test video: `media/vtest.avi` (download it once from `https://github.com/opencv/opencv/raw/4.x/samples/data/vtest.avi`).

Terminal 2 — dashboard

```
git clone https://github.com/adam9798/argus-dashboard
cd argus-dashboard
npm install
printf 'NEXT_PUBLIC_API_BASE_URL=http://localhost:8000\nAUTH_COOKIE_SECURE=false\n' > .env.local
npm run dev
```

Open `http://localhost:3000`, log in as `admin` / `argus-demo`.

> **The dashboard loads events once, when the page opens.** It doesn't poll. Refresh to see new events.

What to show

1. Status strip: live detection FPS, backend, model version.
2. Event list fills as people cross the video.
3. Click an event → **Confirm** or **Correct**. Correct accepts plain English — "false alarm", "suspicious person", "delivery driver" — and the operator's words are kept alongside the label.

A2 · The built-in live page (nothing to install)

The node serves its own page. Same origin as the API, so no CORS, no separate server, no npm:

```
http://<node-address>:8000/dashboard
```

It shows the annotated live feed, **detection FPS refreshed every second**, the backend and model version, how many objects are in frame right now, and recent events with the still saved for each one. It is the fastest way to confirm "it is running and it saw me" from any machine that can reach the node.

The raw feed on its own, with boxes burned in, is at `http://<node-address>:8000/api/camera/stream`.

> The Next.js dashboard in section A is the product UI: login, event review, Confirm/Correct labelling. This page is the operator's view of the live node. They read the same API.

What gets saved

Kind	Where	Recorded as
A still per event, taken at the object's largest	`DATA_DIR/snapshots/`	`media_path` on the `detection` row, served at `/api/events/{id}/snapshot`
A clip per trigger, with pre- and post-roll	`MEDIA_DIR/events/`	a `clip` row, listed at `/api/clips`
Sustained motion	—	a `motion` row, chunked every 60 s

Clips are off unless `RECORD_EVENTS=true` (`--record` for the mock demo). Stills are always kept: they cost about 30 KB each and are what make an event reviewable rather than just a confidence number.

B · On the Pi (screen mirroring, two terminal windows)

Close every terminal first.

Terminal 1 — the node (paste the whole block)

```
cd ~/argus
git fetch origin && git checkout feat/threat-detection && git pull
source .venv/bin/activate
pip install -r requirements.txt
python scripts/export_cpu_models.py --format ncnn
export NODE_NAME="Node A"
export CAMERA_ZONE="Front door"
export AUTH_REQUIRED=false
export FL_ENABLED=false
export THREAT_ENABLED=false
export CORS_ORIGINS="http://localhost:3000"
python main.py
```

The NCNN export takes about a minute once and is picked up automatically on every later start. Leave this window open.

Terminal 2 — check it

```
cd ~/argus
source .venv/bin/activate
curl -s localhost:8000/api/status; echo
curl -s localhost:8000/api/detection/status; echo
```

In `/api/detection/status`, look at `worker`:

Field	Healthy
`running`	`true`
`camera_error`	`null`
`capture_fps`	close to the camera rate
`detect_fps`	above 0 and climbing

You no longer need to open the video stream for detection to run. Open `http://localhost:8000/api/camera/stream` in Chromium only if you want to watch the boxes.

C · Federated round with the validation gate

Needs labelled samples. Seed a synthetic corpus alongside what the camera captured (seeded rows are tagged synthetic and never appear in the dashboard):

```
curl -sX POST localhost:8000/api/federated/samples/bootstrap \
     -H 'Content-Type: application/json' -d '{"site":"driveway","n":4000}'
```

Train locally first — separates a model problem from a network problem:

```
curl -sX POST localhost:8000/api/federated/head/train \
     -H 'Content-Type: application/json' -d '{"epochs":6}'
```

Aggregator (laptop, or a second terminal):

```
python sim/fl.py serve --rounds 3 --clients 1 --address 0.0.0.0:8080
```

Join the round (on the node):

```
curl -sX POST localhost:8000/api/federated/round \
     -H 'Content-Type: application/json' \
     -d '{"server":"<aggregator-ip>:8080","epochs":4}'
```

Watch the result:

```
curl -s localhost:8000/api/federated/round
```

When `running` goes false, `result` holds the gate's decision:

Field	Meaning
`accepted`	`true` → the averaged model is now live and saved
`current.balanced_accuracy`	the model that was running
`candidate.balanced_accuracy`	the model the server sent back
`reason`	why it was rejected, if it was
`label`	new model version, e.g. `fl-r2`, also shown in the dashboard

A rejection is the gate working, not a failure. A non-IID average can suit other sites and hurt this one; the gate keeps the working model.

For real averaging, add a second site as another client:

```
python sim/fl.py serve --rounds 3 --clients 2 --address 0.0.0.0:8080
python sim/fl.py client --site backdoor --address <aggregator-ip>:8080
```

The round trains in its own process on one core, so detection keeps running throughout.

C2 · A federated trial with nothing but the Pi

The aggregator does not have to live on another machine. For a trial, run it on the node itself — three shells, all on the Pi.

Shell 2 — the aggregator:

```
cd ~/argus && source .venv/bin/activate
python sim/fl.py serve --rounds 3 --clients 1 --address 127.0.0.1:8080
```

Shell 3 — seed, train, then join the round:

```
cd ~/argus && source .venv/bin/activate
curl -sX POST localhost:8000/api/federated/samples/bootstrap \
     -H 'Content-Type: application/json' -d '{"site":"driveway","n":4000}'
curl -sX POST localhost:8000/api/federated/head/train \
     -H 'Content-Type: application/json' -d '{"epochs":6}'
curl -sX POST localhost:8000/api/federated/round \
     -H 'Content-Type: application/json' \
     -d '{"server":"127.0.0.1:8080","epochs":4}'
```

Then watch the round and the node together:

```
curl -s localhost:8000/api/federated/round | python -m json.tool
curl -s localhost:8000/api/detection/status | python -m json.tool
```

The round trains in its own process on one core, so `detect_fps` should barely move. `model_version` on `/api/status` becomes `fl-r1`, `fl-r2` and so on each time the gate accepts an update, and the built-in page shows it.

Do not claim that from the architecture — measure it. See **C5**.

C4 · A real second node, on the laptop

`sim/fl.py client` is a simulated client: it replays a generated corpus. It proves the averaging code runs, not that two cameras can disagree and be reconciled. A real second node is one ARGUS install with its own camera, its own sample store and its own name, and the laptop can be one without buying anything — `CAMERA_SOURCE` takes a webcam index or a video file.

LAPTOP — terminal 1 (the second node, "Node B"):

```
cd C:\Users\colab\argus\argus
$env:ARGUS_DATA_DIR = "C:\Users\colab\argus\nodeB"
$env:CAMERA_SOURCE  = "media/vtest.avi"
$env:NODE_NAME      = "Node B"
$env:CAMERA_ZONE    = "Street"
$env:PORT           = "8001"
$env:AUTH_REQUIRED  = "false"
$env:FL_ENABLED     = "false"
$env:RECORD_EVENTS  = "false"
.\.venv\Scripts\python.exe main.py
```

`ARGUS_DATA_DIR` is the part that matters: without it both nodes write one sample store and FedAvg averages a node with itself again, which is the thing this is meant to fix. A separate port is only so the two APIs can coexist on one machine; on the Pi, Node A stays on 8000.

Swap `CAMERA_SOURCE` for `$env:CAMERA_INDEX = "0"` to use the laptop's webcam, which makes the two nodes genuinely see different scenes.

LAPTOP — terminal 2 (give it samples, then join the round):

```
curl.exe -sX POST http://127.0.0.1:8001/api/federated/samples/bootstrap `
  -H "Content-Type: application/json" -d '{\"site\":\"yard\",\"n\":4000}'
curl.exe -sX POST http://127.0.0.1:8001/api/federated/round `
  -H "Content-Type: application/json" `
  -d '{\"server\":\"10.0.0.239:8080\",\"epochs\":4}'
```

Seed Node B from a *different* site than Node A — `yard` against Node A's `driveway` — or the two updates are near-identical and the average has nothing to reconcile. Sites are `driveway`, `backdoor` and `yard`; any other name is a 400 listing those three.

Then start the aggregator with `--clients 2` and have the Pi join as well, so one round averages two stores. Two nodes is still short of the three the Byzantine filters need; say so.

C5 · What training costs the detector

Training is meant to be free — the local head trains in a worker thread, a round runs as its own process. On the laptop that cost 0.1%. The Pi has four cores and no spare headroom, so measure it there rather than assuming it carries over.

PI — terminal 2 (with the node already running in terminal 1):

```
cd ~/argus && source .venv/bin/activate
python scripts/measure_training_cost.py --epochs 400 --baseline 20
python scripts/measure_training_cost.py --mode round --baseline 20
```

It samples `/api/detection/status` before, during and after, and prints each phase's median. Read `capture_fps` for the verdict and `detect_ms` for CPU contention; `detect_fps` follows the motion gate, so it tracks what is in front of the camera, not the training. Keep the scene steady for the whole run.

Two things it refuses to let you misread: a training window too short to sample is reported as "not measurable" rather than as a number, and a change smaller than the baseline's own spread is labelled as such.

Measured on the laptop against a looped clip (18.3s of training, 70 samples): `capture_fps` held at 10.00 throughout, every change inside the baseline spread. That validates the method only — a file source plays at the clip's own rate, so it cannot show headroom. **The Pi number is still to be taken.**

C3 · Which YOLO to load

`OBJECT_MODEL` selects the weights: `yolov8n` (default), `yolov8s`, `yolov8m`. Bigger is more accurate and slower, and an accelerated backend needs an export built at that name:

```
python scripts/export_cpu_models.py --format ncnn --model yolov8s
OBJECT_MODEL=yolov8s python main.py
```

Without an export it falls back to plain PyTorch weights, downloading them on first use. Check what actually loaded:

```
curl -s localhost:8000/api/detection/status | python -m json.tool
```

`model` and `backend` report what is running. On the Pi's CPU, start at `yolov8n`; `yolov8s` is roughly 2–3× the work per frame. The measured ceiling on the Hailo is far higher — `yolov8m` ran at 76 FPS on the AI HAT+ 2 — but that needs the HEF compile.

The four classes

ARGUS class	What triggers it
person	person
vehicle	bicycle, car, motorcycle, bus, train, truck
animal	bird, cat, dog, horse, sheep, cow
package	backpack, handbag, suitcase

COCO has no parcel class, so a doorstep delivery detects as one of the carried-item classes. Everything else COCO knows is dropped before NMS.

D · Configuration

Variable	Default	Purpose
`DETECTION_AUTOSTART`	`true`	Continuous background detection
`CAMERA_SOURCE`	*(board camera)*	Video file path or capture index
`CAMERA_SOURCE_REALTIME`	`true`	Pace a file to its own frame rate
`ARGUS_DATA_DIR`	`data`	Relocates DB, samples, and model state together
`OBJECT_BACKEND`	`auto`	`auto` uses an NCNN/OpenVINO export if present; `pt` forces PyTorch
`OBJECT_IMGSZ`	`640`	YOLO input size
`MOTION_WIDTH`	`320`	Motion detection working width
`STREAM_MAX_FPS` / `STREAM_WIDTH`	`15` / `960`	Viewer stream rate and encode size
`FACE_RECOGNITION_ENABLED`	`false`	Deprecated stage; also needs `requirements-faces.txt`
`ARGUS_TRAIN_THREADS`	`1`	CPU threads a federated round may use

E · If it breaks

Symptom	Cause	Fix
`worker.camera_error: camera failed to initialize`	Pi can't open the camera	Run `rpicam-hello --list-cameras` and `python -c "from picamera2 import Picamera2"`; see next rows
picamera2 import mentions **numpy**	pip pulled numpy 2, which breaks the apt camera library	`pip install --ignore-installed "numpy>=2.2,<3"` then restart Terminal 1
`rpicam-hello` lists no cameras	Ribbon cable	Power off, reseat both ends
`Device or resource busy`	Another process has the camera	Close it; restart Terminal 1
`backend: cpu` on the Pi	No NCNN export	`python scripts/export_cpu_models.py --format ncnn`
People look blue on the live feed	Red and blue channels exchanged	Should be right by default; if your sensor really does hand back RGB, set `CAMERA_SWAP_RB=true` and restart
Dashboard status "offline", curl works	CORS	`CORS_ORIGINS` must include `http://localhost:3000`
Event list stale	Dashboard loads once	Refresh the page
Round returns 409	Too few labelled samples	Bootstrap (section C)
Round never finishes	Aggregator unreachable	Firewall on 8080, wrong IP, or server not started first

F · Measured performance

x86 dev machine **pinned to 4 physical cores** (the Pi 5's core count), on `media/vtest.avi` — a pedestrian scene with motion in 98% of frames, the worst case for a motion-gated pipeline. 300 frames per run, same gating as the live pipeline. Accuracy is agreement with PyTorch at 640 on the same frames (IoU ≥ 0.5, same class).

Change	Before	After	Accuracy
Face recognition off, native res	0.86 FPS	**23.4 FPS**	identity removed, detection unchanged
Face recognition off, 1080p	1.54 FPS	**17.8 FPS**	same
Motion on a 320 px copy, 1080p	16.7 ms motion · 17.8 FPS	**2.6 ms · 22.9 FPS**	boxes still in full-res coordinates (tested)
YOLO input 416 (PyTorch)	22.1 FPS	**35.4 FPS**	recall 0.947 · precision 0.973
YOLO input 320 (PyTorch)	22.1 FPS	37.6 FPS	recall 0.872 · precision 0.987
OpenVINO export @640	22.1 FPS	24.6 FPS	0.998 · 0.998
NCNN export @640	22.1 FPS	16.8 FPS	0.996 · 0.998

How to read it:

- **Face recognition was the dominant cost** — about a second per frame when a person was present, and its thread pool starved YOLO (40 → 117 ms).
- **Motion downscaling matters at camera resolution, not at 768 px.** At native video size motion was already 3 ms; at 1080p it was 30% of the frame.
- **Input size is the biggest remaining lever.** 416 gains ~60% for ~5% recall on this scene. Stay at 640 unless the Pi misses the frame-rate target.
- **NCNN is an ARM runtime.** It loses to PyTorch on x86, which says nothing about the Pi. Measure it there before choosing.
- **Exported models have a fixed input size.** An OpenVINO export built at 640 and asked for 416 returned zero detections — which looked like 211 FPS. The node now uses the export's own size and logs a warning. To change size, re-export: `python scripts/export_cpu_models.py --format ncnn --imgsz 416`.
- **These are x86 numbers.** A Pi 5 core is several times slower. The *relative* gains from removing faces and downscaling motion carry over; absolute FPS must be measured on the Pi.

Recommended Pi configuration now: faces off (default), motion 320 (default), NCNN export, `OBJECT_IMGSZ` 640. If the Pi misses 10 FPS, re-export NCNN at 416.

Known gaps

- **Hailo compile still pending.** ARGUS can't use the Hailo-10H until a YOLO `.hef` is compiled for it (x86 Linux + Hailo Dataflow Compiler, `scripts/export_hailo.py --hw-arch hailo10h`). The Hailo *hardware* is verified: 76 FPS on `yolov8m_h10.hef` with `hailortcli benchmark`. This is **not** blocked on hardware we do not have: the dev laptop is x86_64, so `wsl --install -d Ubuntu-22.04` gives the DFC the Linux host it needs. What is still missing is the DFC itself, which is a manual download behind a free developer.hailo.ai account, and confirmation that the installed DFC version supports `hailo10h`.

- **No vehicle or animal samples.** Balanced accuracy cannot pass 0.75 while two of the five classes are empty. Fill them from footage rather than by hand: `scripts/ingest_video.py street.mp4 --label-by-class` labels each event from the class the detector assigned it, which is what mixed footage needs. Review the result in the dashboard before training — it only ever writes `routine_*`, never `anomaly`.
- **Threat model on CPU is expensive.** Keep `THREAT_ENABLED=false` until it is compiled for the accelerator.
- **Dashboard auth mismatch.** The dashboard never sends its token to the backend. Fine at `AUTH_REQUIRED=false` on a LAN; must be fixed before any tunnel.